

```

0001: /*****
0002: * Project      : Autonomous Drone Fleet Management System (ADFMS)
0003: * Module       : Flight Controller
0004: * Version      : 1.3.2
0005: *
0006: * Description:
0007: * This software controls autonomous medical delivery drones.
0008: * It authenticates operators, validates missions,
0009: * executes drone commands and records mission activities.
0010: *
0011: * NOTE:
0012: * This software is intentionally developed as part of a
0013: * Secure Software Code Review laboratory exercise.
0014: *
0015: * Organization:
0016: * SkyFleet Technologies
0017: *
0018: *****/
0019:
0020: #include <iostream>
0021: #include <string>
0022: #include <cstring>
0023: #include <ctime>
0024:
0025: using namespace std;
0026:
0027: /*****
0028: * USER CLASS
0029: *****/
0030: class User
0031: {
0032: private:
0033:
0034:     string username;
0035:     string password;
0036:     string role;
0037:
0038: public:
0039:
0040:     User(string u,string p)
0041:     {
0042:         username = u;
0043:         password = p;
0044:
0045:         // Default role assigned after login
0046:         role = "Operator";
0047:     }
0048:
0049:     string getUsername() const
0050:     {
0051:         return username;
0052:     }
0053:
0054:     string getPassword() const
0055:     {
0056:         return password;
0057:     }
0058:
0059:     string getRole() const
0060:     {
0061:         return role;
0062:     }
0063:
0064:     void setRole(string r)
0065:     {
0066:         role = r;
0067:     }
0068: };
0069:
0070: /*****
0071: * AUTHENTICATION CLASS
0072: *****/
0073: class Authentication
0074: {
0075: private:
0076:
0077:     string validUser = "pilot1";
0078:     string validPassword = "Drone@123";
0079:
0080: public:
0081:
0082:     bool authenticate(User& user);
0083:
0084:     bool login(User& user);
0085:
0086:     string getRole(User& user);
0087: };
0088:
0089: bool Authentication::authenticate(User& user)
0090: {
0091:     if(user.getUsername()==validUser &&
0092:        user.getPassword()==validPassword)
0093:     {
0094:         return true;
0095:     }
0096:
0097:     return false;
0098:

```

```

0099: }
0100:
0101: bool Authentication::login(User& user)
0102: {
0103:     cout<<"Authenticating Operator..."<<endl;
0104:
0105:     if(authenticate(user))
0106:     {
0107:         cout<<"Login Successful"<<endl;
0108:         return true;
0109:     }
0110:
0111:     cout<<"Authentication Failed"<<endl;
0112:
0113:     return false;
0114: }
0115:
0116: string Authentication::getRole(User& user)
0117: {
0118:     return user.getRole();
0119: }
0120:
0121: /*****
0122:  * MISSION CLASS
0123:  *****/
0124: class Mission
0125: {
0126: private:
0127:
0128:     int missionID;
0129:     string destination;
0130:     double altitude;
0131:     double payloadWeight;
0132:
0133: public:
0134:
0135:     Mission(int id,
0136:             string dest,
0137:             double alt,
0138:             double payload);
0139:
0140:     bool validateMission();
0141:
0142:     int getMissionID() const;
0143:
0144:     string getDestination() const;
0145:
0146:     double getAltitude() const;
0147:
0148:     double getPayloadWeight() const;
0149: };
0150:
0151:
0152: Mission::Mission(int id,
0153:                  string dest,
0154:                  double alt,
0155:                  double payload)
0156: {
0157:     missionID=id;
0158:     destination=dest;
0159:     altitude=alt;
0160:     payloadWeight=payload;
0161: }
0162:
0163: bool Mission::validateMission()
0164: {
0165:     if(altitude<=0)
0166:         return false;
0167:
0168:     if(payloadWeight<0)
0169:         return false;
0170:
0171:     return true;
0172: }
0173:
0174: int Mission::getMissionID() const
0175: {
0176:     return missionID;
0177: }
0178:
0179: string Mission::getDestination() const
0180: {
0181:     return destination;
0182: }
0183:
0184: double Mission::getAltitude() const
0185: {
0186:     return altitude;
0187: }
0188:
0189: double Mission::getPayloadWeight() const
0190: {
0191:     return payloadWeight;
0192: }
0193:
0194: /*****
0195:  * LOGGER CLASS
0196:  *****/

```

```

0197: class Logger
0198: {
0199: public:
0200:
0201:     Logger();
0202:
0203:     void logEvent(const string& eventType,
0204:                  const string& username,
0205:                  const string& message);
0206:
0207:     void logError(const string& message);
0208:
0209:     string currentTime();
0210: };
0211:
0212:
0213: Logger::Logger()
0214: {
0215: }
0216:
0217: string Logger::currentTime()
0218: {
0219:     time_t now=time(0);
0220:
0221:     char* dt=ctime(&now);
0222:
0223:     return string(dt);
0224: }
0225:
0226: void Logger::logEvent(const string& eventType,
0227:                      const string& username,
0228:                      const string& message)
0229: {
0230:     cout<<endl;
0231:     cout<<"-----"<<endl;
0232:     cout<<"Time : "<<currentTime();
0233:     cout<<"Type : "<<eventType<<endl;
0234:     cout<<"User : "<<username<<endl;
0235:     cout<<"Info : "<<message<<endl;
0236:     cout<<"-----"<<endl;
0237: }
0238:
0239: void Logger::logError(const string& message)
0240: {
0241:     cout<<endl;
0242:     cout<<"-----"<<endl;
0243:     cout<<"Time : "<<currentTime();
0244:     cout<<"ERROR : "<<message<<endl;
0245:     cout<<"-----"<<endl;
0246: }
0247:
0248: /*****
0249:  * FLIGHT CONTROLLER CLASS
0250:  *****/
0251: class FlightController
0252: {
0253: private:
0254:
0255:     Authentication auth;
0256:     Logger logger;
0257:
0258:     bool missionActive;
0259:
0260: public:
0261:
0262:     FlightController();
0263:
0264:     bool executeMission(User& user,
0265:                        Mission& mission);
0266:
0267:     bool authorizeMission(User& user);
0268:
0269:     bool validateMission(Mission& mission);
0270:
0271:     int calculatePayload(Mission& mission);
0272:
0273:     bool sendMission(Mission& mission);
0274:
0275:     void emergencyStop();
0276:
0277:     void getTelemetry();
0278:
0279:     bool abortMission(string reason);
0280: };
0281:
0282: /*****
0283:  * FlightController Implementation
0284:  *****/
0285:
0286: FlightController::FlightController()
0287: {
0288:     missionActive = false;
0289: }
0290:
0291: /*****
0292:  * Verify user authorization.
0293:  *****/
0294: bool FlightController::authorizeMission(User& user)

```

```

0295: {
0296:     if(user.getRole() == "Operator")
0297:     {
0298:         return true;
0299:     }
0300:
0301:     return false;
0302: }
0303:
0304: /*****
0305:  * Validate mission.
0306:  *****/
0307: bool FlightController::validateMission(Mission& mission)
0308: {
0309:     return mission.validateMission();
0310: }
0311:
0312: /*****
0313:  * Calculate payload.
0314:  *****/
0315: int FlightController::calculatePayload(Mission& mission)
0316: {
0317:     int totalPayload;
0318:
0319:     totalPayload = mission.getPayloadWeight();
0320:
0321:     // Packaging weight
0322:     totalPayload += 5;
0323:
0324:     return totalPayload;
0325: }
0326:
0327: /*****
0328:  * Upload mission to drone.
0329:  *****/
0330: bool FlightController::sendMission(Mission& mission)
0331: {
0332:     char destination[20];
0333:
0334:     strcpy(destination,
0335:             mission.getDestination().c_str());
0336:
0337:     cout << endl;
0338:     cout << "Uploading Mission..." << endl;
0339:
0340:     cout << "Mission ID   : "
0341:            << mission.getMissionID()
0342:            << endl;
0343:
0344:     cout << "Destination  : "
0345:            << destination
0346:            << endl;
0347:
0348:     cout << "Altitude     : "
0349:            << mission.getAltitude()
0350:            << " meters" << endl;
0351:
0352:     missionActive = true;
0353:
0354:     return true;
0355: }
0356:
0357: /*****
0358:  * Emergency Stop
0359:  *****/
0360: void FlightController::emergencyStop()
0361: {
0362:     missionActive = false;
0363:
0364:     cout << endl;
0365:     cout << "*** EMERGENCY STOP ACTIVATED ***"
0366:            << endl;
0367: }
0368:
0369: /*****
0370:  * Retrieve telemetry.
0371:  *****/
0372: void FlightController::getTelemetry()
0373: {
0374:     cout << endl;
0375:
0376:     cout << "===== Drone Telemetry ====="
0377:            << endl;
0378:
0379:     cout << "Battery       : 91%" << endl;
0380:     cout << "GPS           : Locked" << endl;
0381:     cout << "Altitude      : Stable" << endl;
0382:
0383:     cout << "Mission       : ";
0384:
0385:     if(missionActive)
0386:         cout << "ACTIVE";
0387:     else
0388:         cout << "IDLE";
0389:
0390:     cout << endl;
0391: }
0392:

```

```

0393: /*****
0394:  * Abort Mission
0395:  *****/
0396: bool FlightController::abortMission(string reason)
0397: {
0398:     cout << endl;
0399:     cout << "Mission Aborted." << endl;
0400:
0401:     cout << "Reason : "
0402:         << reason
0403:         << endl;
0404:
0405:     emergencyStop();
0406:
0407:     return false;
0408: }
0409:
0410: /*****
0411:  * Complete Mission Execution Pipeline
0412:  *****/
0413: bool FlightController::executeMission(User& user,
0414:                                     Mission& mission)
0415: {
0416:     cout << endl;
0417:     cout << "===== " << endl;
0418:     cout << " AUTONOMOUS DRONE MISSION EXECUTION " << endl;
0419:     cout << "===== " << endl;
0420:
0421:     //-----
0422:     // Step 1 : Authenticate
0423:     //-----
0424:
0425:     if(!auth.authenticate(user))
0426:     {
0427:         logger.logError("Authentication failed for user : "
0428:                        + user.getUsername());
0429:
0430:         return false;
0431:     }
0432:
0433:     cout << "Authentication Successful." << endl;
0434:
0435:     //-----
0436:     // Step 2 : Authorization
0437:     //-----
0438:
0439:     if(!authorizeMission(user))
0440:     {
0441:         logger.logError("Authorization Failed.");
0442:
0443:         return false;
0444:     }
0445:
0446:     cout << "Authorization Successful." << endl;
0447:
0448:     //-----
0449:     // Step 3 : Mission Validation
0450:     //-----
0451:
0452:     if(!validateMission(mission))
0453:     {
0454:         logger.logError("Mission Validation Failed.");
0455:
0456:         return false;
0457:     }
0458:
0459:     cout << "Mission Validation Successful."
0460:         << endl;
0461:
0462:     //-----
0463:     // Step 4 : Payload Calculation
0464:     //-----
0465:
0466:     int payload = calculatePayload(mission);
0467:
0468:     cout << "Verified Payload : "
0469:         << payload
0470:         << " kg"
0471:         << endl;
0472:
0473:     //-----
0474:     // Step 5 : Upload Mission
0475:     //-----
0476:
0477:     if(!sendMission(mission))
0478:     {
0479:         logger.logError("Mission Upload Failed.");
0480:
0481:         return false;
0482:     }
0483:
0484:     //-----
0485:     // Step 6 : Telemetry
0486:     //-----
0487:
0488:     getTelemetry();
0489:
0490:     //-----

```

```

0491: // Step 7 : Completion
0492: //-----
0493:
0494: cout << endl;
0495: cout << "Mission Completed Successfully."
0496:     << endl;
0497:
0498: return true;
0499: }
0500:
0501:
0502: /*****
0503:  * MAIN PROGRAM
0504:  *****/
0505: int main()
0506: {
0507:     cout << endl;
0508:     cout << "===== " << endl;
0509:     cout << " Autonomous Drone Fleet Management System " << endl;
0510:     cout << "===== " << endl;
0511:
0512:     User operator1("pilot1",
0513:                   "Drone@123");
0514:
0515:     Mission mission1(
0516:         1001,
0517:         "CityHospital",
0518:         120,
0519:         18.5);
0520:
0521:     FlightController controller;
0522:
0523:     if(controller.executeMission(operator1,
0524:                                 mission1))
0525:     {
0526:         cout << endl;
0527:         cout << "Mission Execution Successful."
0528:             << endl;
0529:     }
0530:     else
0531:     {
0532:         cout << endl;
0533:         cout << "Mission Execution Failed."
0534:             << endl;
0535:     }
0536:
0537:     return 0;
0538: }

```